

Instruction / Bits		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Description	N	Z	C	V
Logic, Word (16-bit)																						
NOT	rd, rs	0	0	0	0	0			rd	X	X		rs		X	X	X	rd = ! rs	*	*	-	-
XOR	rd, rs, rt	0	0	0	0	1			rd	X	X		rs				rt	rd = rs ^ rt	*	*	-	-
OR	rd, rs, rt	0	0	0	1	0			rd	X	X		rs				rt	rd = rs rt	*	*	-	-
AND	rd, rs, rt	0	0	0	1	1			rd	X	X		rs				rt	rd = rs & rt	*	*	-	-
Logic, Bit operations (Mask Bit, Invert Bit, Set Bit, Clear Bit)																						
MKB	rd, rs, B	1	0	0	0	0			rd	X	B3		rs				B2..0	rd = rs & (1 <<B)	*	*	-	-
INB	rd, rs, B	1	0	0	0	1			rd	X	B3		rs				B2..0	rd = rs ^ (1 <<B)	*	*	-	-
SEB	rd, rs, B	1	0	0	1	0			rd	X	B3		rs				B2..0	rd = rs (1 <<B)	*	*	-	-
CLB	rd, rs, B	1	0	0	1	1			rd	X	B3		rs				B2..0	rd = rs & ~(1 << B)	*	*	-	-
Shift and roll																						
SHL	rd, rs	0	0	1	0	0			rd	X	X		rs		X	X	X	rd = rs <<1	*	*	*	*
SHR	rd, rs	0	0	1	0	1			rd	X	X		rs		X	X	X	rd = rs >>1	*	*	*	*
ROL	rd, rs	1	0	1	0	0			rd	X	X		rs		X	X	X	rd = (rs[14..0], Carry)	*	*	*	*
ROR	rd, rs	1	0	1	0	1			rd	X	X		rs		X	X	X	rd = (Carry, rs[15..1])	*	*	*	*
Arithmetic (without/with Carry)																						
SUB	rd, rs, rt	0	0	1	1	0			rd	X	X		rs				rt	rd = rs - rt	*	*	*	*
ADD	rd, rs, rt	0	0	1	1	1			rd	X	X		rs				rt	rd = rs + rt	*	*	*	*
SBB	rd, rs, rt	1	0	1	1	0			rd	X	X		rs				rt	rd = rs - rt - Carry	*	*	*	*
ADC	rd, rs, rt	1	0	1	1	1			rd	X	X		rs				rt	rd = rs + rt + Carry	*	*	*	*
Load register with immediate data (8 bit)																						
LDL	rd, #0xYY	x	1	0	0	0			rd				- DATA (8 bit) -					rd = (rd & 0xFF00) 0x00YY	-	-	-	-
LDH	rd, #0xYY	x	1	0	0	1			rd				- DATA (8 bit) -					rd = (rd & 0x00FF) 0xYY00	-	-	-	-
Internal Memory																						
STO	[rs], rt	0	1	0	1	0	0	0	0	X			rs				rt	Mem[rs] = rt	-	-	-	-
LDD	rd, [rs]	0	1	0	1	1			rd	0	X		rs		X	X	X	rd = Mem[rs]	-	-	-	-
External Memory or I/O																						
OUT	[rs], rt	0	1	0	1	0	0	0	0	1	X		rs				rt	IO[rs] = rs	-	-	-	-
IN	rd, [rs]	0	1	0	1	1			rd	1	X		rs		X	X	X	rd = IO[rs]	-	-	-	-
Stack (Stack space occupies last 16 or 32 addresses in data memory)																						
PSH	rt	1	1	0	1	0	X	X	X	X	X	X	X	X			rt	Mem[SP+1] = rt, SP++	-	-	-	-
POP	rd	1	1	0	1	1			rd	X	X	X	X	X	X	X	X	rd = Mem[SP], SP--	-	-	-	-
Jumps																						
JZ	NewPC	0	1	1	0	0				- NewPC (11 bit) -								PC = NewPC if Zero	-	-	-	-
JC	NewPC	0	1	1	0	1				- NewPC (11 bit) -								PC = NewPC if Carry	-	-	-	-
JNZ	NewPC	1	1	1	0	0				- NewPC (11 bit) -								PC = NewPC if not Zero	-	-	-	-
JNC	NewPC	1	1	1	0	1				- NewPC (11 bit) -								PC = NewPC if not Carry	-	-	-	-
JS	NewPC	0	1	1	1	0				- NewPC (11 bit) -								PSH PC+1, PC = NewPC	-	-	-	-
JMP	NewPC	1	1	1	1	0				- NewPC (11 bit) -								PC = NewPC (unconditional)	-	-	-	-
BRA	BCOND, OffsetPC	1	1	1	1	1			BCOND				- OffsetPC (8 bit) -					PC += OffsetPC if BCOND	-	-	-	-
RTS		0	1	1	1	1	0	X	X	0	X	X	X	X	X	X	w	POP PC	-	-	-	-
RTI		0	1	1	1	1	1	X	X	1	X	X	X	X	X	X	w	POP PC, C, Z, N, V	*	*	*	*
Interrupts (Aliases)																						
SEI		x	1	0	0	0	0	0	1	X	X	X	X	X	X	X	1	Enable interrupts (LDL r1,#1)	-	-	-	-
CLI		x	1	0	0	0	0	0	1	X	X	X	X	X	X	X	0	Disable interrupts (LDL r1,#0)	-	-	-	-
Aliases																						
NOP		x	1	0	0	0	0	0	0	X	X	X	X	X	X	X	X	No operation (LDL r0, #XX)	-	-	-	-
HLT		1	1	1	1	1	0	X	X	0	0	0	0	0	0	0	0	Halt the program (BRA 0XX 0)	-	-	-	-

Flags: N (Negative), Z (Zero), C (Carry), V (Overflow); * change, - no change
BCOND: condition for BRA, 0XX unconditional, 100 if Z=1, 101 if C=1, 110 if V=1, 111 if N=1
OffsetPC: 8 bit two's complement
Registers: R0=0, R1=1, R2 ... R7 (16-bit)
Power Up: 0 → PC; 0 → N, Z, C, V; 0 → R2..R7; 0 → SP; Disable interrupts
IRQ: C & Z & N & V & NextPC → Stack; 1 → PC; no further IRQ possible until RTI